Binary Search

AlgoExpert · Easy · Searching

The problem

- Return the index of target in a sorted array, or -1.

Intuition

- In a sorted array, comparing with the middle element tells us which half can contain the target.

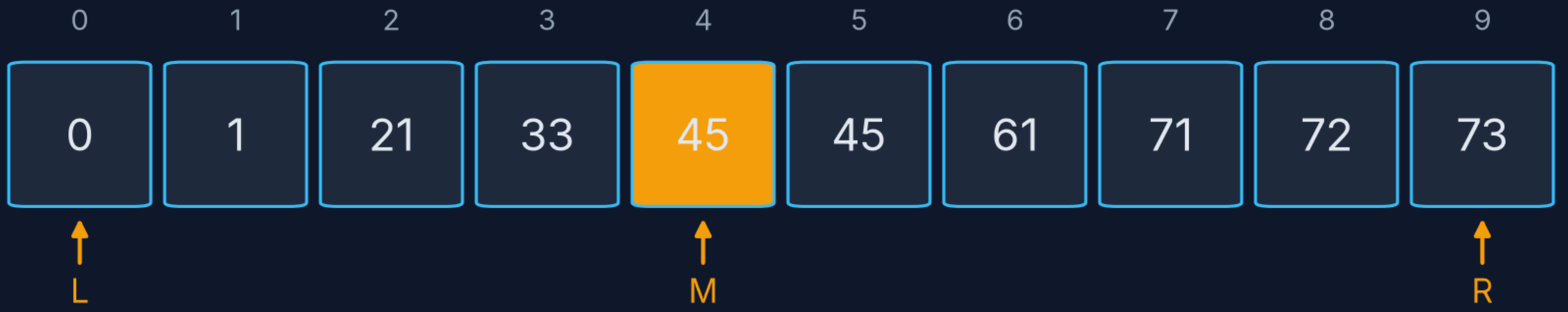
Brute force vs optimal

- Linear scan: O(n)
- Binary search: halve the range each step → O(log n)
- 1,000,000 items → about 20 comparisons

Approach

- 1. Keep left and right bounds
- 2. Check the middle
- 3. Discard the half that can't contain the target
- 4. Return the index or minus one

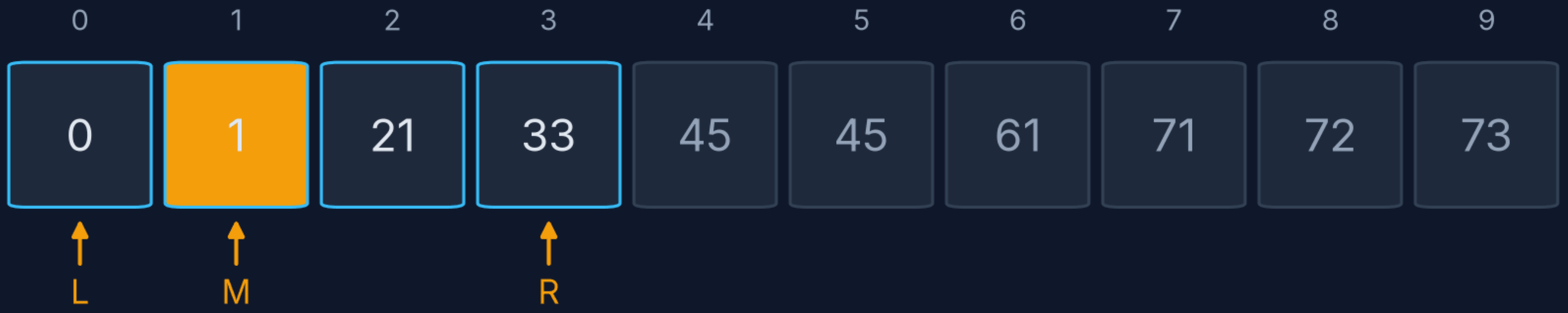
Worked example




```
target = 33
a[4] = 45
```

45 > 33 → search left half

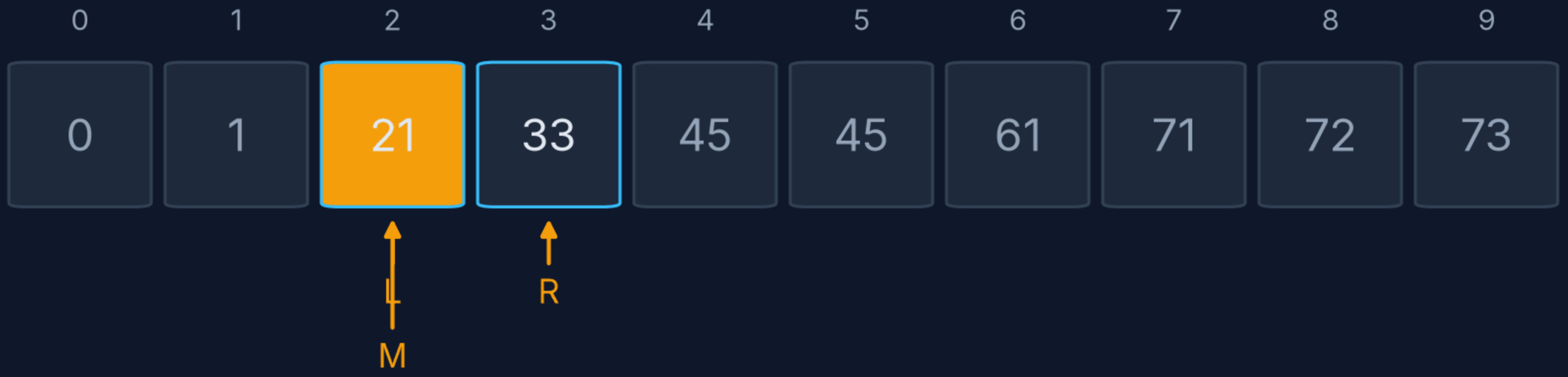
Worked example




```
target = 33
a[1] = 1
```

1 < 33 → search right half

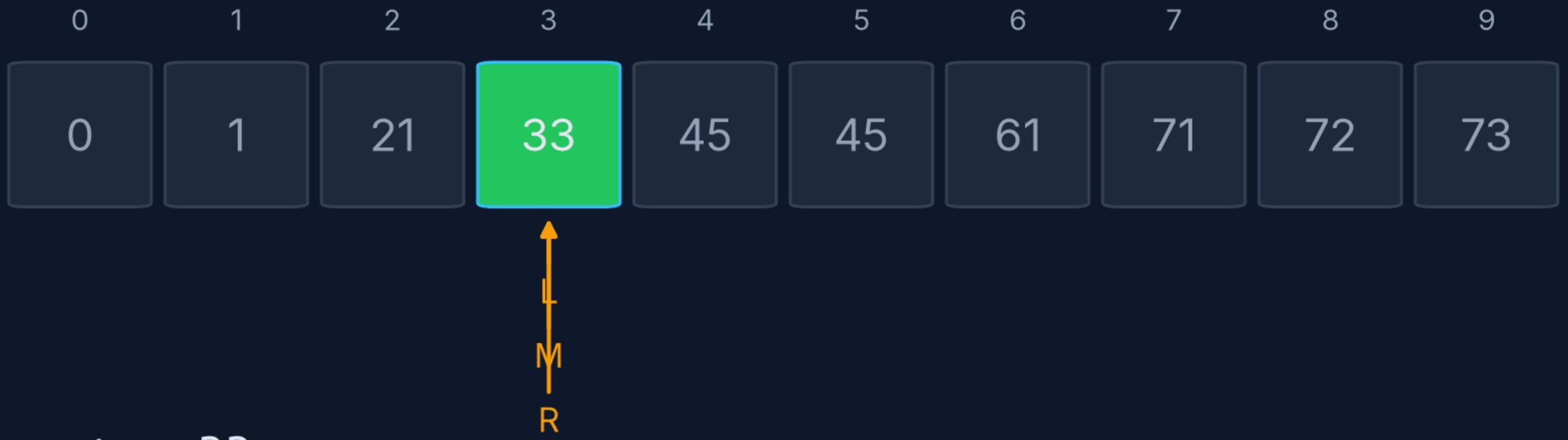
Worked example




```
target = 33
a[2] = 21
```

21 < 33 → search right half

Worked example




```
target = 33
a[3] = 33
```

a[3] = 33 → found

Java solution

```java
public class BinarySearch {
    public static int binarySearch(int[] array, int target) {
        int lo = 0, hi = array.length - 1;
        while (lo <= hi) {
            int mid = (lo + hi) >>> 1;
            if (array[mid] == target) return mid;
            if (array[mid] < target) lo = mid + 1; else hi = mid - 1;
        }
        return -1;
    }
}
```

Complexity

- Time: O(log n)
- Space: O(1)

Edge cases & pitfalls

- Loop condition is left ≤ right
- Mid as left + (right − left) / 2 avoids overflow
- Not found → −1

Interview tips & follow-ups

- Know recursive + iterative
- Follow-ups: first/last occurrence, rotated arrays, search on answer

Recap

- In a sorted array, comparing with the middle element tells us which half can contain the target.
- Complexity: O(log n)